

22 — SQLite Grundlagen

Datenbanken mit Python: connect, CREATE, INSERT, SELECT

HTW Berlin – Wirtschaftsingenieurwesen

SoSe 2026

Agenda

1. Warum SQLite? – eine Datenbank ohne Server
2. Der immer gleiche Ablauf: connect → cursor → execute → commit/fetch → close
3. CREATE TABLE – Struktur festlegen
4. INSERT – Daten einfügen (sicher, mit Platzhaltern)
5. SELECT – Daten abfragen
6. Saubere Patterns: with, Fehlerbehandlung, sqlite3.Row

Lernziel: Mit Python eine SQLite-Datenbank anlegen, befüllen und abfragen – und SQL-Injection vermeiden.

Wie wir heute arbeiten: Konzept → *Live-Demo* (ich im Terminal, am Veggie-Soles-Shop) → *Sofort ausprobieren* (Sie im Notebook 22).

Warum SQLite?

- In Notebook 12 haben wir Daten in **Dateien** geschrieben – aber gezielt suchen, filtern, verknüpfen? Mühsam.
- MySQL/PostgreSQL können das – brauchen aber einen Server, Installation, Konfiguration.
- **SQLite**: die ganze Datenbank ist *eine Datei*. Kein Server, keine Einrichtung. In Python schon eingebaut (`import sqlite3`).
- Steckt in Firefox, Chrome, Android, iOS – ideal für kleine bis mittlere Anwendungen, Prototypen, Lehre.

```
import sqlite3  
conn = sqlite3.connect("veggie_soles.db")    # legt die Datei an,
```

Statt eines Dateinamens geht auch `:memory:` – eine Datenbank, die nur im Arbeitsspeicher lebt (gut zum Ausprobieren).

Der immer gleiche Ablauf

SQLite mit Python — der immer gleiche Ablauf



Benutzereingaben NIE in den String kleben — immer Platzhalter: `execute("... VALUES (?, ?)", (name, preis))`

Noch besser: `with sqlite3.connect('shop.db') as conn:` → schließt automatisch, auch bei Fehlern

blau = reines Python · orange = hier steckt SQL drin

► **Live-Demo** - 01_verbindung_und_tabelle.py

✎ **Sofort ausprobieren** - Notebook 22, Übung 1.1: Verbindung zu `uebung.db`, Cursor, schließen.

CREATE TABLE – die Struktur festlegen

```
cursor.execute("""
    CREATE TABLE IF NOT EXISTS produkt (
        produkt_id INTEGER PRIMARY KEY,
        name        TEXT NOT NULL,
        preis        REAL,
        kategorie    TEXT
    )
""")
conn.commit()
```

- der SQL-Befehl ist ein **String**, mit `execute()` abgeschickt – ohne `commit()` ist nichts gespeichert
- `IF NOT EXISTS`: kein Fehler, wenn die Tabelle schon da ist
- `INTEGER PRIMARY KEY` zählt automatisch hoch · `NOT NULL` macht eine Spalte zur Pflicht

INSERT - Daten einfügen

```
# SICHER: Werte als Platzhalter (?), nicht in den String kleben!  
cursor.execute("INSERT INTO produkt (name, preis) VALUES (?, ?)",
```

```
# Viele auf einmal:
```

```
cursor.executemany("INSERT INTO produkt (name, preis) VALUES (?,  
                                [ ("Hemp-High", 109.00), ("Bambus-Boot", 135.50  
conn.commit()
```

- `cursor.lastrowid` - ID der zuletzt eingefügten Zeile ·
`cursor.rowcount` - Anzahl betroffener Zeilen
- **Goldene Regel:** Benutzereingaben *nie* per f-string/+ in SQL - nur Platzhalter ?. Sonst SQL-Injection.

SELECT - Daten abfragen

```
cursor.execute("SELECT name, preis FROM produkt WHERE preis < ?",  
for name, preis in cursor.fetchall():  
    print(name, preis)
```

execute() schickt nur ab – die Ergebnisse **holt** man danach:

fetchone()	fetchall()	fetchmany(n)
eine Zeile (oder None)	alle Zeilen als Liste von Tupeln	die nächsten n Zeilen

► **Live-Demo** - 03_abfragen_select.py

✎ **Sofort ausprobieren** – Notebook 22, Übung 4.1: alle Studierenden aus „Informatik“ (nur Vor- und Nachname).

Saubere Patterns

```
with sqlite3.connect("shop.db") as conn:  
    conn.execute(sql, daten)
```

```
try:  
    conn.execute(sql_mit_unique_spalte, daten)  
except sqlite3.IntegrityError:  
    print("Wert verletzt UNIQUE / NOT NULL")
```

- **with:** am Blockende automatisch `commit()` – bei einer Exception `rollback()`
- **try/except `sqlite3.IntegrityError`:** gezielt fangen, was schiefgehen kann (Dublette, NULL in NOT NULL ...)
- **`conn.row_factory = sqlite3.Row`:** Zeilen wie ein Dict ansprechen
– `zeile["name"]` statt `zeile[1]`

Cheat Card - Notebook 22

Aufgabe	Code
Verbindung	<code>conn = sqlite3.connect("shop.db")</code>
Cursor	<code>cur = conn.cursor()</code>
Tabelle anlegen	<code>cur.execute("CREATE TABLE IF NOT EXISTS ...")</code>
Einfügen (sicher)	<code>cur.execute("INSERT ... VALUES (?, ?)", (a, b))</code>
viele einfügen	<code>cur.executemany("INSERT ...", liste_von_tupeln)</code>
abfragen	<code>cur.execute("SELECT ... WHERE x = ?", (wert,))</code>
Ergebnisse	<code>cur.fetchone() / cur.fetchall()</code>
sichern / schließen	<code>conn.commit() · conn.close() · oder with sqlite3.connect(...) as conn:</code>

Ausblick: Notebook 23 - SQLite Vertiefung

```
SELECT  kunde.name, SUM(bestellung.betrag) AS umsatz
FROM    kunde JOIN bestellung ON kunde.kunde_id = bestellung.kun
WHERE   bestellung.betrag > 50
GROUP BY kunde.kunde_id
HAVING  umsatz > 200
ORDER BY umsatz DESC;
```

- **WHERE** feiner: AND/OR, BETWEEN, IN, LIKE, IS NULL · **ORDER BY** – sortieren
- **JOIN** – Tabellen verbinden (INNER, LEFT) · **UPDATE** / **DELETE** – ändern und löschen
- **GROUP BY** + Aggregatfunktionen – Auswertungen

Heute gelernt

- ✓ SQLite = Datenbank in einer Datei, in Python eingebaut
- ✓ Der Ablauf: connect → cursor → execute → commit/fetch → close
- ✓ CREATE TABLE mit den fünf SQLite-Datentypen
- ✓ INSERT – sicher mit Platzhaltern, executemany für viele
- ✓ SELECT – fetchone/fetchall, WHERE
- ✓ with, try/except, sqlite3.Row

Zur Vertiefung im Notebook 22:

- „Sofort ausprobieren“-Übungen in Kap. 1-5 (sind Sie schon mitgegangen)
- Abschlussübungen Aufg. 1-4: eigene Datenbank `firma.db`, `get_high_earners`, `Notenverwaltung`, `backup_table`